

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO9 – Develop a Program Using Dictionary

Nominal Duration: Theory 3.5 Hours / Practical 9 Hours

L.O 09: DEVELOP A PROGRAM USING DICTIONARY

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Explain the properties and characteristics of Python dictionaries.
2. Declare and initialize Python dictionaries.
3. Create and print dictionary data.
4. Identify dictionary keys and values.
5. Access dictionary elements using keys.
6. Access dictionary elements using the `get()` method.
7. Retrieve dictionary keys using `keys()`.
8. Retrieve dictionary values using `values()`.
9. Retrieve dictionary key-value pairs using `items()`.
10. Change the value of a specific dictionary item.
11. Add new key-value pairs to a dictionary.
12. Update dictionary information using `update()`.
13. Remove dictionary items using appropriate methods and keywords.
14. Apply common dictionary methods.
15. Apply membership keywords such as `in` and `not in`.
16. Create dictionaries using the `dict()` constructor.
17. Create and access nested dictionaries.
18. Compare dictionaries with other Python data structures.
19. Apply dictionaries to structured data-storage problems.
20. Develop, test, and debug Python programs using dictionaries.

INFORMATION SHEET No. 9.1

Title: Understanding and Using Python Dictionaries

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain Python dictionary characteristics.
- Describe dictionary syntax and structure.
- Create dictionaries.
- Explain key-value pairs.
- Access dictionary values using keys.
- Use `get()` to access dictionary information.
- Retrieve keys, values, and items.
- Add and modify dictionary items.
- Remove dictionary items.
- Apply common dictionary methods.
- Apply dictionary-related keywords.
- Create dictionaries using `dict()`.
- Create nested dictionaries.
- Compare dictionaries with lists, tuples, and sets.
- Apply dictionaries to practical data-management problems.

1. Introduction to Python Dictionaries

A dictionary is a Python data structure used to store information as **key-value pairs**.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}
```

The dictionary contains three items:

```
"name"    → "Dara"  
"age"     → 20  
"course"  → "Python"
```

Each item contains:

- A **key**
- A **value**

2. Dictionary Characteristics

Python dictionaries have several important characteristics.

A dictionary:

- Stores data as key-value pairs.
- Is changeable or mutable.
- Does not use numeric positional indexing like lists and tuples.
- Uses keys to access values.
- Requires keys to be unique.
- Can contain values of different data types.
- Can contain nested dictionaries.
- Can grow or shrink after creation.
- Preserves insertion order in current Python 3 versions.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "score": 85.5,  
    "active": True  
}
```

Values can include:

- String
- Integer
- Float
- Boolean
- List
- Tuple
- Dictionary
- Other Python objects

3. Dictionary Syntax

A dictionary is commonly declared using curly braces:

```
{}
```

General syntax:

```
dictionary_name = {  
    key1: value1,  
    key2: value2,  
    key3: value3  
}
```

Example:

```
person = {  
    "name": "Sokha",  
    "age": 22,  
    "city": "Phnom Penh"  
}
```

4. Creating an Empty Dictionary

An empty dictionary can be created using:

```
student = {}
```

Check its type:

```
print(type(student))
```

Output:

```
<class 'dict'>
```

5. Creating a Dictionary Using dict()

Python also provides the dict() constructor.

Example:

```
student = dict(  
    name="Dara",  
    age=20,  
    course="Python"  
)  
  
print(student)
```

Output:

```
{'name': 'Dara', 'age': 20, 'course': 'Python'}
```

6. Key-Value Pairs

A dictionary item contains a key and its associated value.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}
```

Here:

Key	Value
"name"	"Dara"
"age"	20

The colon : separates each key from its value.

Commas separate dictionary items.

7. Dictionary Keys

Keys identify dictionary values.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}
```

The keys are:

```
name  
age  
course
```

Keys must be unique within the same dictionary.

Example:

```
student = {  
    "name": "Dara",  
    "name": "Sokha"  
}
```

The later value replaces the earlier value for the duplicate key.

Therefore, dictionary keys should be unique.

8. Dictionary Values

Dictionary values store the actual information.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "score": 85.5  
}
```

The values are:

```
Dara  
20  
85.5
```

Unlike keys, dictionary values may be duplicated.

9. Printing a Dictionary

Use `print()` to display the complete dictionary.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}  
  
print(student)
```

Output:

```
{'name': 'Dara', 'age': 20, 'course': 'Python'}
```

10. Accessing Dictionary Elements

Dictionary values can be accessed using their keys.

Syntax:

```
dictionary[key]
```

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}  
  
print(student["name"])
```

Output:

Dara

Another example:

```
print(student["course"])
```

Output:

Python

11. Accessing a Missing Key

If a key does not exist:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
print(student["email"])
```

Python produces:

KeyError

because "email" does not exist.

12. Using get()

The get() method can retrieve a dictionary value.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
print(student.get("name"))
```

Output:

Dara

One advantage of `get()` is that a missing key does not normally produce a `KeyError`.

Example:

```
print(student.get("email"))
```

Output:

None

13. Default Value with `get()`

A default value can be provided.

Example:

```
print(student.get("email", "Not Available"))
```

Output:

Not Available

This is useful when a key may not exist.

14. Retrieving Dictionary Keys

The `keys()` method returns the dictionary keys.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}
```

```
print(student.keys())
```

Output resembles:

```
dict_keys(['name', 'age', 'course'])
```

15. Retrieving Dictionary Values

The `values()` method returns dictionary values.

Example:

```
print(student.values())
```

Output resembles:

```
dict_values(['Dara', 20, 'Python'])
```

16. Retrieving Dictionary Items

The `items()` method returns dictionary items as key-value pairs.

Example:

```
print(student.items())
```

Output resembles:

```
dict_items([('name', 'Dara'), ('age', 20), ('course', 'Python')])
```

17. Accessing Keys and Values Separately

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}  
  
print("Keys:", student.keys())  
print("Values:", student.values())  
print("Items:", student.items())
```

This allows a program to inspect the dictionary structure.

18. Changing a Dictionary Value

Dictionaries are mutable.

A specific value can be changed using its key.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}  
  
student["age"] = 21  
  
print(student)
```

Output:

```
{'name': 'Dara', 'age': 21, 'course': 'Python'}
```

19. Changing Another Dictionary Item

Example:

```
student["course"] = "AI Basic Python Developer"  
  
print(student)
```

The value associated with "course" is updated.

20. Adding a New Dictionary Item

A new key-value pair can be added using:

```
dictionary[new_key] = value
```

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
student["course"] = "Python"  
  
print(student)
```

Output:

```
{'name': 'Dara', 'age': 20, 'course': 'Python'}
```

21. update() Method

The update() method can add or modify dictionary items.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
student.update({"age": 21})  
  
print(student)
```

Output:

```
{'name': 'Dara', 'age': 21}
```

Add a new item:

```
student.update({"course": "Python"})
```

Output:

```
{'name': 'Dara', 'age': 21, 'course': 'Python'}
```

22. Updating Multiple Items

The update() method can update several items at once.

Example:

```
student.update({  
    "age": 22,  
    "course": "AI Basic Python Developer",  
    "level": "II"  
})  
  
print(student)
```

23. Removing Dictionary Items

Python provides several ways to remove dictionary items.

Common methods and keywords include:

```
pop()
popitem()
del
clear()
```

24. pop() Method

pop() removes an item by its key and returns the removed value.

Example:

```
student = {
    "name": "Dara",
    "age": 20,
    "course": "Python"
}

removed_value = student.pop("age")

print(student)
print("Removed:", removed_value)
```

Output:

```
{'name': 'Dara', 'course': 'Python'}
Removed: 20
```

25. popitem() Method

popitem() removes and returns the most recently inserted key-value pair.

Example:

```
student = {
    "name": "Dara",
    "age": 20,
    "course": "Python"
}

removed_item = student.popitem()

print(student)
print(removed_item)
```

26. del Keyword

The del keyword can remove an item using its key.

Example:

```
student = {
    "name": "Dara",
    "age": 20,
    "course": "Python"
}

del student["age"]

print(student)
```

Output:

```
{'name': 'Dara', 'course': 'Python'}
```

27. clear() Method

clear() removes all dictionary items.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
student.clear()  
  
print(student)
```

Output:

```
{}
```

The dictionary still exists but contains no items.

28. Checking Whether a Key Exists

Use the in keyword.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20  
}  
  
print("name" in student)
```

Output:

```
True
```

29. Using not in

Example:

```
print("email" not in student)
```

Output:

```
True
```

By default, membership testing on a dictionary checks the keys.

30. Conditional Check for a Key

Example:

```
if "course" in student:  
    print("Course information exists")
```

This is useful before accessing optional data.

31. Dictionary Methods

Common dictionary methods include:

Method	Purpose
keys()	Return keys
values()	Return values
items()	Return key-value pairs
get()	Retrieve value safely
update()	Add/update items
pop()	Remove item by key
popitem()	Remove last inserted item
clear()	Remove all items
copy()	Create a shallow copy
setdefault()	Get value or create key with default

The methods most directly required by LO9 are:

```
keys()
values()
items()
get()
update()
pop()
```

32. copy() Method

The copy() method creates a shallow copy of a dictionary.

Example:

```
student = {
    "name": "Dara",
    "age": 20
}

student_copy = student.copy()

print(student_copy)
```

Output:

```
{'name': 'Dara', 'age': 20}
```

33. setdefault() Method

setdefault() returns the value of a key if the key exists.

If the key does not exist, it creates the key with a default value.

Example:

```
student = {  
    "name": "Dara"  
}  
  
student.setdefault("course", "Python")  
  
print(student)
```

Output:

```
{'name': 'Dara', 'course': 'Python'}
```

34. Dictionary Length

The `len()` function determines the number of key-value pairs.

Example:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}  
  
print(len(student))
```

Output:

```
3
```

35. Dictionary Constructor

A dictionary can be created with `dict()`.

Example:

```
student = dict(  
    name="Dara",  
    age=20,  
    course="Python"  
)
```

Display:

```
print(student)
```

Output:

```
{'name': 'Dara', 'age': 20, 'course': 'Python'}
```

36. Dictionaries Containing Lists

A dictionary value can be a list.

Example:

```
student = {  
    "name": "Dara",  
    "scores": [80, 85, 90]  
}
```

```
print(student["scores"])
```

Output:

```
[80, 85, 90]
```

Access one score:

```
print(student["scores"][0])
```

Output:

```
80
```

37. Dictionaries Containing Tuples

Example:

```
student = {  
    "name": "Dara",  
    "subjects": ("Python", "Database", "AI")  
}  
  
print(student["subjects"][1])
```

Output:

```
Database
```

38. Nested Dictionaries

A nested dictionary is a dictionary containing another dictionary.

Example:

```
students = {  
    "student1": {  
        "name": "Dara",  
        "score": 85  
    },  
    "student2": {  
        "name": "Sokha",  
        "score": 90  
    }  
}
```

The main dictionary contains two student dictionaries.

39. Accessing Nested Dictionary Data

Given:

```
students = {  
    "student1": {  
        "name": "Dara",  
        "score": 85  
    },  
    "student2": {  
        "name": "Sokha",  
        "score": 90  
    }  
}
```

```
}  
}
```

Access Dara's name:

```
print(students["student1"]["name"])
```

Output:

Dara

Access Sokha's score:

```
print(students["student2"]["score"])
```

Output:

90

40. Changing Nested Dictionary Data

Example:

```
students["student1"]["score"] = 88
```

```
print(students["student1"])
```

Output:

```
{'name': 'Dara', 'score': 88}
```

41. Adding Data to a Nested Dictionary

Example:

```
students["student1"]["course"] = "Python"
```

```
print(students["student1"])
```

Output:

```
{'name': 'Dara', 'score': 88, 'course': 'Python'}
```

42. Dictionary vs List

Feature	Dictionary	List
Syntax	{key: value}	[value1, value2]
Access	By key	By numeric index
Mutable	Yes	Yes
Duplicate keys/items	Keys must be unique	Duplicate items allowed
Best for	Structured named data	Ordered sequences

Example dictionary:

```
student = {  
    "name": "Dara",  
    "age": 20  
}
```

Example list:

```
student = ["Dara", 20]
```

The dictionary is often clearer because each value has a descriptive key.

43. Dictionary vs Tuple

Feature	Dictionary	Tuple
Syntax	{key: value}	(item1, item2)
Mutable	Yes	No
Access	Keys	Indexes
Data model	Key-value pairs	Ordered sequence

44. Dictionary vs Set

Feature	Dictionary	Set
Stores	Key-value pairs	Unique values
Syntax	{"name": "Dara"}	{"Python", "Java"}
Access by key	Yes	No
Duplicate keys/elements	Keys unique	Elements unique

45. Practical Application — Student Record

A dictionary is suitable for storing a student record.

Example:

```
student = {
    "id": "ST001",
    "name": "Dara",
    "gender": "Male",
    "age": 20,
    "course": "AI Basic Python Developer",
    "score": 85
}
```

Each key clearly describes its associated value.

46. Practical Application — Product Record

```
product = {
    "id": "P001",
    "name": "Keyboard",
    "price": 25.50,
    "quantity": 10
}
```

Access:

```
print(product["name"])
print(product["price"])
```

47. Practical Application — Configuration Data

```
settings = {
    "language": "English",
}
```

```
    "theme": "Dark",
    "notifications": True
}
```

Dictionaries are useful when information has meaningful field names.

48. Basic Dictionary Program

```
student = {
    "name": "Dara",
    "age": 20,
    "course": "Python"
}

print("Student:", student)

print("Name:", student["name"])
print("Age:", student["age"])

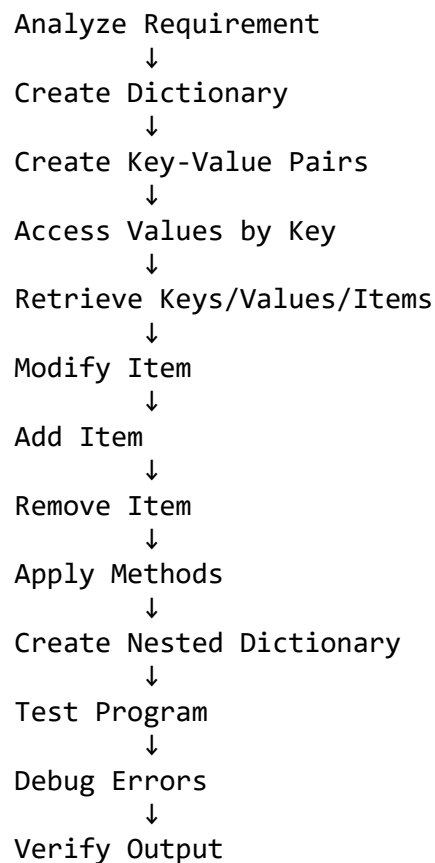
print("Keys:", student.keys())
print("Values:", student.values())

student["age"] = 21
student["level"] = "II"

print("Updated:", student)
```

49. Dictionary Operation Process

A dictionary program can follow this process:



50. Common Dictionary Errors

Missing Key

Incorrect:

```
student = {  
    "name": "Dara"  
}  
  
print(student["age"])
```

Result:

KeyError

One solution:

```
print(student.get("age", "Not Available"))
```

Incorrect Dictionary Syntax

Incorrect:

```
student = {  
    "name" = "Dara",  
    "age" = 20  
}
```

Correct:

```
student = {  
    "name": "Dara",  
    "age": 20  
}
```

Dictionary keys and values are separated by a colon :.

Missing Comma

Incorrect:

```
student = {  
    "name": "Dara"  
    "age": 20  
}
```

Correct:

```
student = {  
    "name": "Dara",  
    "age": 20  
}
```

Incorrect Method

Incorrect:

```
student.delete("age")
```

Correct examples:

```
student.pop("age")
```

or:

```
del student["age"]
```

51. Debugging Dictionary Programs

When a dictionary program produces an error:

1. Check curly braces.
2. Check colons between keys and values.
3. Check commas between items.
4. Check quotation marks.
5. Check whether the key exists.
6. Check key spelling.
7. Remember that keys are case-sensitive.
8. Check dictionary method names.
9. Check method arguments.
10. Check nested dictionary levels.
11. Compare expected and actual values.
12. Correct the program.
13. Run it again.
14. Verify the final output.

SELF-CHECK No. 9.1

Part A — Multiple Choice

1. Which symbols are commonly used to create a dictionary?

- A. [] B. () C. {} D. <>

2. A dictionary stores data as:

- A. Rows only B. Key-value pairs C. Indexes only D. Characters only

3. Which character separates a key from its value?

- A. , B. : C. ; D. =

4. How do you access the value associated with "name"?

- A. student[0] B. student("name") C. student["name"] D. student.name()

5. Which method can retrieve a value safely?

- A. get() B. find() C. read() D. value()

6. Which method retrieves dictionary keys?

- A. key() B. keys() C. getkeys() D. names()

7. Which method retrieves dictionary values?

- A. value() B. values() C. getvalues() D. items()

8. Which method returns key-value pairs?

- A. items() B. pairs() C. data() D. all()

9. Which method adds or modifies dictionary items?

- A. append() B. update() C. insert() D. extend()

10. Which method removes an item using a key?

- A. pop() B. append() C. index() D. sort()

11. Which keyword can remove a dictionary item?

- A. remove B. del C. delete D. erase

12. Which function returns the number of dictionary items?

- A. size() B. count() C. len() D. length()

13. Dictionary keys should normally be:

- A. Duplicate B. Unique C. Lists only D. Numbers only

14. A dictionary containing another dictionary is called:

- A. String dictionary B. Nested dictionary
C. Tuple dictionary D. Sequence dictionary

15. Python dictionaries are:

- A. Immutable B. Mutable C. Always empty D. Numeric only

ANSWER KEY No. 9.1

Part A

No.	Answer
1	C
2	B
3	B
4	C
5	A
6	B
7	B
8	A
9	B
10	A
11	B
12	C
13	B
14	B
15	B

OPERATION SHEET No. 9.1

Title: Create, Access, and Manipulate Python Dictionaries

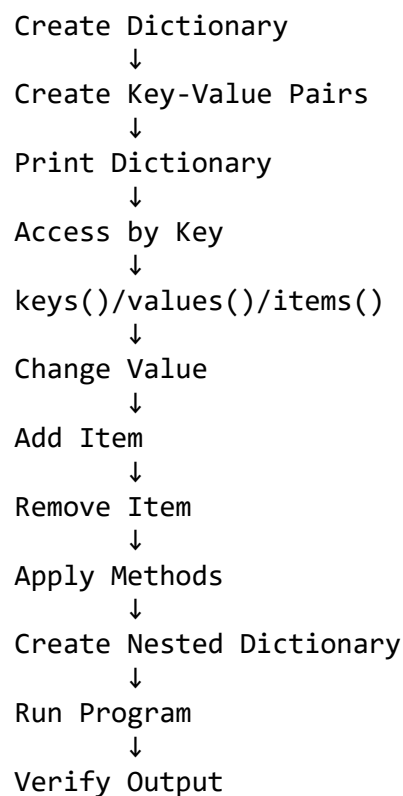
Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must create a dictionary, access elements by key, retrieve keys and values, modify information, add and remove items, apply dictionary methods, create a nested dictionary, execute the program, and verify the output.

Description

This operation demonstrates the standard procedure for creating and manipulating Python dictionary data.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/VS Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Guide	Copy	1
7	Python Dictionary Exercise File	Set	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	Development environment opens successfully
2	Create new Python file	Source file is created
3	Declare dictionary	Correct {key: value} syntax is used
4	Print dictionary	Complete dictionary is displayed
5	Access value using key	Correct value is retrieved
6	Apply get()	Required value is retrieved safely
7	Apply keys()	Dictionary keys are returned
8	Apply values()	Dictionary values are returned
9	Apply items()	Key-value pairs are returned
10	Change specific value	Required value is updated
11	Add new item	New key-value pair is inserted
12	Apply update()	Dictionary is updated correctly
13	Apply pop()	Required item is removed
14	Apply membership keyword	Key presence is correctly checked
15	Create nested dictionary	Multi-level data is structured correctly
16	Save program	.py extension is used
17	Execute program	Program runs successfully
18	Verify output	Results meet requirements

Operation Procedure

Create:

```
student = {  
    "id": "ST001",  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}
```

Print:

```
print(student)
```

Access:

```
print(student["name"])  
print(student.get("course"))
```

Retrieve dictionary information:

```
print(student.keys())  
print(student.values())  
print(student.items())
```

Change:

```
student["age"] = 21
```

Add:

```
student["level"] = "II"
```

Update:

```
student.update({"course": "AI Basic Python Developer"})
```

Remove:

```
student.pop("age")
```

Check membership:

```
print("name" in student)
```

Create nested dictionary:

```
students = {
    "student1": {
        "name": "Dara",
        "score": 85
    },
    "student2": {
        "name": "Sokha",
        "score": 90
    }
}

print(students["student1"]["name"])
```

Save as:

```
dictionary_operations.py
```

Execute and verify the results.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 9.1

Title: Create, Print, and Access Python Dictionaries

Performance Task

Create a Python dictionary, print its contents, and access specified dictionary values using keys.

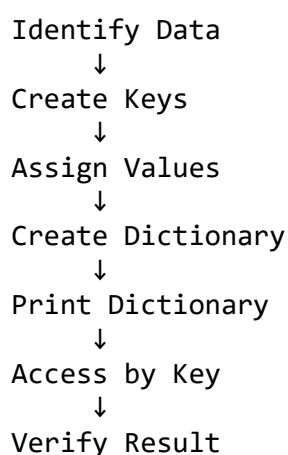
Performance Objective

Given Python IDLE/IDE and programming requirements, the trainee must correctly create a dictionary, print it, and retrieve specified values.

Description

The trainee practices dictionary declaration, key-value pairs, printing, and basic element access.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Dictionary Exercise File	Set	1
5	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create dictionary	Correct syntax is applied
2	Create keys	Keys are meaningful and unique
3	Assign values	Values are correctly associated
4	Print dictionary	Complete dictionary is displayed
5	Access "name"	Correct value is retrieved

No.	Procedure/Step	Key Performance
6	Access "age"	Correct value is retrieved
7	Apply get()	Value is retrieved correctly
8	Verify output	Results match requirements

Practical Exercise

Create:

```
student = {
    "id": "ST001",
    "name": "Dara",
    "age": 20,
    "course": "Python"
}
```

Display:

```
print(student)
print(student["name"])
print(student["age"])
print(student.get("course"))
```

Record:

Expression	Expected Result	Actual Result
student["name"]	Dara	_____
student["age"]	20	_____
student.get("course")	Python	_____

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 9.2

Title: Apply Dictionary Keys, Values, and Items

Performance Task

Retrieve dictionary keys, values, and key-value pairs using built-in dictionary methods.

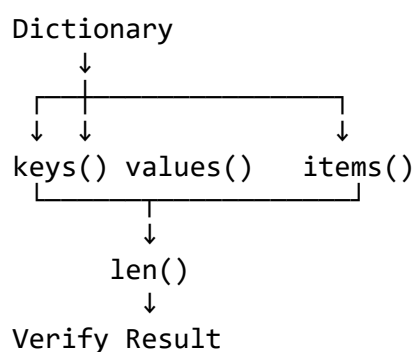
Performance Objective

Given a dictionary, the trainee must correctly apply `keys()`, `values()`, `items()`, and `len()`.

Description

The trainee practices inspecting dictionary structure and content.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	1. Create dictionary	Correct data is stored
2	2. Apply <code>keys()</code>	Keys are correctly returned
3	3. Apply <code>values()</code>	Values are correctly returned
4	4. Apply <code>items()</code>	Key-value pairs are returned
5	5. Apply <code>len()</code>	Number of items is correct
6	6. Verify results	Output matches dictionary contents

Practical Exercise

```
product = {  
    "id": "P001",  
    "name": "Keyboard",
```

```

    "price": 25.50,
    "quantity": 10
}

print("Keys:", product.keys())
print("Values:", product.values())
print("Items:", product.items())
print("Length:", len(product))

```

Complete:

Operation	Result
keys()	_____
values()	_____
items()	_____
len(product)	_____

Assessment Method

- Direct observation
- Practical exercise
- Worksheet review
- Output verification

TASK SHEET No. 9.3

Title: Change and Add Dictionary Items

Performance Task

Modify existing dictionary values and add new key-value pairs.

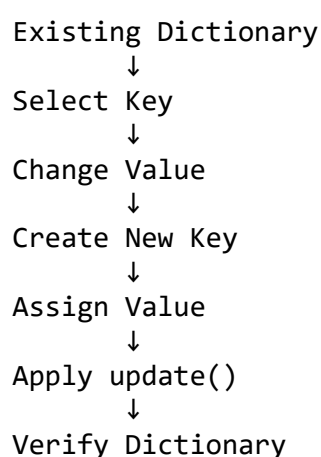
Performance Objective

Given an existing dictionary, the trainee must change specified values and add new items using direct assignment and `update()`.

Description

The trainee practices dictionary modification and expansion.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create dictionary	Original information is correct
2	Select existing key	Correct item is identified
3	Change value	Existing value is updated
4	Add new key	New key is unique
5	Assign new value	Correct value is stored
6	Apply <code>update()</code>	Required data is modified

No.	Procedure/Step	Key Performance
7	Display dictionary	Updated information is correct

Practical Exercise

Create:

```
student = {
    "name": "Dara",
    "age": 20,
    "course": "Python"
}
```

Change age:

```
student["age"] = 21
```

Add level:

```
student["level"] = "II"
```

Update course:

```
student.update({
    "course": "AI Basic Python Developer"
})
```

Display:

```
print(student)
```

Record final dictionary:

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 9.4

Title: Remove Dictionary Items

Performance Task

Remove dictionary items using appropriate dictionary methods and keywords.

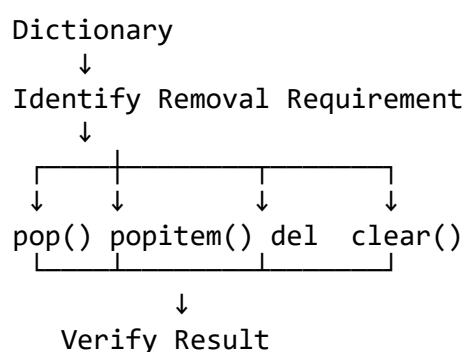
Performance Objective

Given an existing dictionary, the trainee must correctly use `pop()`, `popitem()`, `del`, and `clear()` according to requirements.

Description

The trainee practices dictionary removal operations.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create dictionary	Required items are present
2	Apply <code>pop()</code>	Correct key is removed
3	Apply <code>popitem()</code>	Last inserted pair is removed
4	Apply <code>del</code>	Specified item is removed
5	Apply <code>clear()</code>	All items are removed when required
6	Verify results	Dictionary contents are correct

Practical Exercise

```
student = {  
    "id": "ST001",  
    "name": "Dara",  
    "age": 20,  
    "course": "Python",  
    "level": "II"  
}
```

Remove age:

```
student.pop("age")  
print(student)
```

Remove the most recently inserted item:

```
student.popitem()  
print(student)
```

Remove course:

```
del student["course"]  
print(student)
```

Create a copy and clear it:

```
temp_student = student.copy()  
temp_student.clear()  
  
print(temp_student)
```

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 9.5

Title: Apply Dictionary Methods and Keywords

Performance Task

Apply dictionary methods and membership keywords to retrieve and manage information.

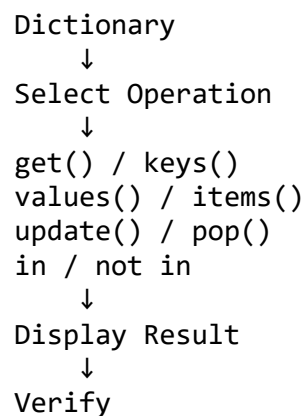
Performance Objective

Given a dictionary, the trainee must correctly apply `get()`, `keys()`, `values()`, `items()`, `update()`, `pop()`, `in`, and `not in`.

Description

This activity consolidates the most important dictionary methods required by LO9.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Python Reference Guide	Copy	1
5	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Apply <code>get()</code>	Correct value/default is returned
2	Apply <code>keys()</code>	Keys are returned
3	Apply <code>values()</code>	Values are returned
4	Apply <code>items()</code>	Pairs are returned
5	Apply <code>update()</code>	Data is updated
6	Apply <code>pop()</code>	Required item is removed

No.	Procedure/Step	Key Performance
7	Apply in	Existing key is identified
8	Apply not in	Missing key is identified
9	Verify result	Operations produce correct output

Practical Exercise

```

product = {
    "id": "P001",
    "name": "Mouse",
    "price": 15.00,
    "quantity": 20
}

print(product.get("name"))
print(product.get("brand", "Not Available"))

print(product.keys())
print(product.values())
print(product.items())

product.update({"price": 17.50})

print("name" in product)
print("brand" not in product)

removed = product.pop("quantity")

print(product)
print("Removed:", removed)

```

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 9.6

Title: Create and Manage Nested Dictionaries

Performance Task

Create a nested dictionary to store several student records and access, modify, and add information within the nested structure.

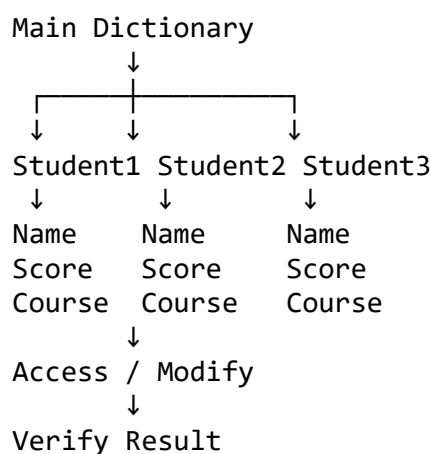
Performance Objective

Given student information, the trainee must organize data into nested dictionaries and correctly access and modify multi-level values.

Description

The trainee practices storing structured records using nested dictionaries.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Dictionary Exercise File	Set	1
5	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create main dictionary	Outer structure is correct
2	Create student dictionaries	Nested records are correctly created
3	Access first student name	Correct nested keys are used
4	Access second student score	Correct value is retrieved
5	Modify nested value	Required information is updated

No.	Procedure/Step	Key Performance
6	Add nested item	New key-value pair is inserted
7	Display final data	Nested information is correct
8	Verify output	Results meet requirements

Practical Exercise

Create:

```
students = {
    "student1": {
        "name": "Dara",
        "score": 85,
        "course": "Python"
    },
    "student2": {
        "name": "Sokha",
        "score": 90,
        "course": "Python"
    },
    "student3": {
        "name": "Vanna",
        "score": 75,
        "course": "Python"
    }
}
```

Display Dara:

```
print(students["student1"]["name"])
```

Display Sokha's score:

```
print(students["student2"]["score"])
```

Change Vanna's score:

```
students["student3"]["score"] = 80
```

Add level to Dara:

```
students["student1"]["level"] = "II"
```

Display:

```
print(students)
```

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

JOB SHEET No. 9.1

Title: Develop a Student Record Management Program Using Dictionaries

Performance Task

Develop a complete Python program that uses dictionaries to create, access, update, remove, and manage structured student information.

Performance Objective

Given a computer, Python IDE, programming requirements, and reference materials, the trainee must create and print dictionaries, access dictionary elements, retrieve keys and values, modify information, add and remove items, apply dictionary methods and keywords, create a nested dictionary, test the program, debug errors, and verify the final output.

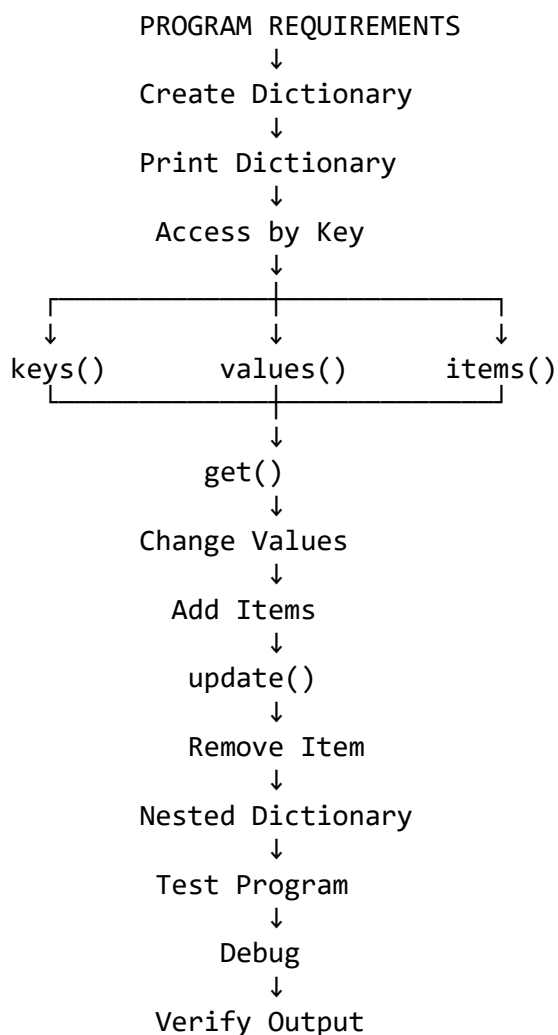
Description

Develop:

`student_dictionary_manager.py`

The program will demonstrate all major dictionary operations required in LO9.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Python Dictionary Exercise Files	Set	1
6	Python Programming Reference Guide	Copy	1
7	Programming Worksheet	Copy	1
8	USB Flash Drive	Unit	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required dictionary operations are identified
2	Open Python IDE	Development environment opens successfully
3	Create .py file	Source file is created
4	Create dictionary	Correct syntax is applied
5	Create key-value pairs	Keys and values are appropriately defined
6	Print dictionary	Complete record is displayed
7	Access elements by key	Correct values are retrieved
8	Apply get()	Required value is retrieved safely
9	Apply keys()	Dictionary keys are displayed
10	Apply values()	Dictionary values are displayed
11	Apply items()	Key-value pairs are displayed
12	Change specific value	Correct item is updated
13	Add dictionary item	New key-value pair is added
14	Apply update()	Multiple/individual values are updated
15	Apply pop()	Required item is removed
16	Apply in/not in	Key membership is correctly evaluated
17	Create nested dictionary	Student records are organized correctly
18	Access nested information	Correct nested keys are used
19	Modify nested data	Required nested value is updated
20	Save program	.py extension is used
21	Execute program	Program runs successfully
22	Test dictionary operations	Required operations are checked
23	Identify errors	Key/syntax errors are recognized
24	Debug program	Errors are corrected
25	Verify final output	Results match requirements

Program Requirements

The program must:

1. Create a student dictionary.
2. Print the complete dictionary.
3. Access student name using its key.
4. Access course using `get()`.
5. Display all keys.
6. Display all values.
7. Display all key-value pairs.
8. Change a specific value.
9. Add a new key-value pair.
10. Apply `update()`.
11. Remove an item using `pop()`.
12. Check whether a key exists.
13. Create at least three student records using a nested dictionary.
14. Access nested dictionary information.
15. Modify a nested value.
16. Print the final data.
17. Test and verify the program.

Part 1 — Create Student Dictionary

Create:

```
student = {  
    "id": "ST001",  
    "name": "Dara",  
    "age": 20,  
    "course": "Python",  
    "score": 85  
}
```

Part 2 — Print Dictionary

```
print(student)
```

Part 3 — Access Dictionary Elements

```
print("Name:", student["name"])  
print("Course:", student.get("course"))
```

Part 4 — Keys, Values, and Items

```
print("Keys:", student.keys())  
print("Values:", student.values())  
print("Items:", student.items())
```

Part 5 — Modify Dictionary

Change age:

```
student["age"] = 21
```

Add level:

```
student["level"] = "II"
```

Update course:

```
student.update({  
    "course": "AI Basic Python Developer"  
})
```

Part 6 — Remove an Item

Remove age:

```
removed_age = student.pop("age")
```

Display:

```
print("Removed Age:", removed_age)
```

Part 7 — Check Keys

```
print("name" in student)  
print("email" not in student)
```

Part 8 — Nested Dictionary

Create:

```
students = {  
    "ST001": {  
        "name": "Dara",  
        "score": 85,  
        "course": "Python"  
    },  
    "ST002": {  
        "name": "Sokha",  
        "score": 90,  
        "course": "Python"  
    },  
    "ST003": {  
        "name": "Vanna",  
        "score": 75,  
        "course": "Python"  
    }  
}
```

Access:

```
print(students["ST001"]["name"])  
print(students["ST002"]["score"])
```

Modify:

```
students["ST003"]["score"] = 80
```

Add:

```
students["ST001"]["level"] = "II"
```

Example Complete Program

```
# Student Dictionary Management Program
```

```

student = {
    "id": "ST001",
    "name": "Dara",
    "age": 20,
    "course": "Python",
    "score": 85
}

print("STUDENT DICTIONARY")
print(student)

print("\nACCESS DATA")
print("Name:", student["name"])
print("Course:", student.get("course"))

print("\nKEYS, VALUES AND ITEMS")
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())

print("\nMODIFY DATA")
student["age"] = 21
student["level"] = "II"

student.update({
    "course": "AI Basic Python Developer",
    "score": 88
})

print(student)

print("\nREMOVE DATA")
removed_age = student.pop("age")
print("Removed age:", removed_age)
print(student)

print("\nMEMBERSHIP")
print("name" in student)
print("email" not in student)

print("\nNESTED DICTIONARY")

students = {
    "ST001": {
        "name": "Dara",
        "score": 85,
        "course": "Python"
    },
    "ST002": {
        "name": "Sokha",
        "score": 90,
        "course": "Python"
    },
    "ST003": {
        "name": "Vanna",
        "score": 75,
        "course": "Python"
    }
}

```

```

    }
}

print("ST001 Name:", students["ST001"]["name"])
print("ST002 Score:", students["ST002"]["score"])

students["ST003"]["score"] = 80
students["ST001"]["level"] = "II"

print("\nUPDATED NESTED DICTIONARY")
print(students)

```

Example Expected Main Output

STUDENT DICTIONARY

```
{'id': 'ST001', 'name': 'Dara', 'age': 20, 'course': 'Python', 'score': 85}
```

ACCESS DATA

Name: Dara

Course: Python

KEYS, VALUES AND ITEMS

Keys: dict_keys(['id', 'name', 'age', 'course', 'score'])

Values: dict_values(['ST001', 'Dara', 20, 'Python', 85])

MODIFY DATA

```
{'id': 'ST001', 'name': 'Dara', 'age': 21, 'course': 'AI Basic Python Developer',
 'score': 88, 'level': 'II'}
```

REMOVE DATA

Removed age: 21

MEMBERSHIP

True

True

NESTED DICTIONARY

ST001 Name: Dara

ST002 Score: 90

UPDATED NESTED DICTIONARY

...

Testing Requirements

Test Item	Expected Result	Actual Result	Pass/Fail
student["name"]	Dara	_____	_____
student.get("course") initially	Python	_____	_____
Updated age	21	_____	_____
Added level	II	_____	_____
Updated score	88	_____	_____
"name" in student	True	_____	_____
"email" not in student	True	_____	_____
students["ST001"]["name"]	Dara	_____	_____

Test Item	Expected Result	Actual Result	Pass/Fail
students["ST002"]["score"]	90	_____	_____
Updated ST003 score	80	_____	_____

Debugging Requirements

Error 1 — Missing Key

Incorrect:

```
student = {
    "name": "Dara"
}

print(student["age"])
```

The trainee must identify the `KeyError`.

Possible correction:

```
print(student.get("age", "Not Available"))
```

Error 2 — Incorrect Dictionary Syntax

Incorrect:

```
student = {
    "name" = "Dara",
    "age" = 20
}
```

Correct:

```
student = {
    "name": "Dara",
    "age": 20
}
```

Error 3 — Incorrect Method

Incorrect:

```
student.append({"course": "Python"})
```

A dictionary does not use `append()` to add an item.

Correct:

```
student["course"] = "Python"
```

or:

```
student.update({"course": "Python"})
```

Error 4 — Incorrect Nested Key

Incorrect:

```
print(students["student1"]["age"])
```

when "age" does not exist.

The trainee must identify the missing key and apply the appropriate correction.

Assessment Method

- Oral questioning
- Written/theoretical assessment
- Practical workshop assessment
- Direct observation
- Source-code inspection
- Program execution
- Output verification
- Debugging demonstration

WORKSHEET No. 9.1

Title: Python Dictionary Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Create a Dictionary

Create a dictionary containing:

- Student ID
- Name
- Age
- Course

```
student = {  
    _____,  
    _____,  
    _____,  
    _____  
}
```

Part B — Identify Keys and Values

Given:

```
student = {  
    "name": "Dara",  
    "age": 20,  
    "course": "Python"  
}
```

Complete:

Key	Value
"name"	_____
"age"	_____
"course"	_____

Part C — Access Dictionary Elements

Write code to display "Dara":

Write code to display 20:

Use get() to display "Python":

Part D — Dictionary Methods

Complete:

Requirement	Method
Return keys	_____
Return values	_____
Return key-value pairs	_____
Retrieve value safely	_____
Add/update information	_____
Remove by key	_____

Part E — Change Dictionary Value

Given:

```
student = {
    "name": "Dara",
    "age": 20
}
```

Change age to 21.

Part F — Add Dictionary Item

Add:

```
course = Python
```

Code:

Part G — update()

Use update() to change:

```
course = AI Basic Python Developer
```

Code:

Part H — Remove Item

Remove "age" using pop().

Part I — Membership

Given:

```
student = {
    "name": "Dara",
    "course": "Python"
}
```

Determine:

Expression	Result
"name" in student	_____
"age" in student	_____
"age" not in student	_____

Part J — Nested Dictionary

Given:

```
students = {
    "ST001": {
        "name": "Dara",
        "score": 85
    },
    "ST002": {
        "name": "Sokha",
        "score": 90
    }
}
```

Write code to display "Dara":

Write code to display 90:

Change Dara's score to 88:

Part K — Compare Data Structures

Complete:

Feature	Dictionary	List	Tuple
Main syntax	_____	_____	_____
Access by	_____	_____	_____
Mutable	_____	_____	_____
Stores key-value pairs	_____	_____	_____

Part L — Debugging

Correct:

```
student = {
    "name" = "Dara",
    "age" = 20
}
```

Correct code:

What is wrong with:

```
student = {  
    "name": "Dara"  
}  
  
print(student["course"])
```

Answer:

How can `get()` be used to avoid the error?

PERFORMANCE CHECKLIST No. 9.1

Title: Develop a Program Using Dictionary

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Created a Python dictionary correctly	☐	☐
2	Applied correct dictionary syntax	☐	☐
3	Created appropriate key-value pairs	☐	☐
4	Used unique and meaningful keys	☐	☐
5	Printed the complete dictionary	☐	☐
6	Accessed dictionary elements using keys	☐	☐
7	Applied <code>get()</code> correctly	☐	☐
8	Identified dictionary keys	☐	☐
9	Identified dictionary values	☐	☐
10	Applied <code>keys()</code> correctly	☐	☐
11	Applied <code>values()</code> correctly	☐	☐
12	Applied <code>items()</code> correctly	☐	☐
13	Determined dictionary length using <code>len()</code>	☐	☐
14	Changed the value of a specific dictionary item	☐	☐
15	Added a new key-value pair	☐	☐
16	Applied <code>update()</code> correctly	☐	☐
17	Updated multiple values where required	☐	☐
18	Applied <code>pop()</code> correctly	☐	☐
19	Applied <code>popitem()</code> appropriately	☐	☐
20	Applied <code>del</code> correctly	☐	☐
21	Applied <code>clear()</code> appropriately	☐	☐
22	Applied <code>copy()</code> appropriately	☐	☐
23	Applied <code>in</code> to check dictionary keys	☐	☐
24	Applied <code>not in</code> correctly	☐	☐
25	Used <code>dict()</code> constructor appropriately	☐	☐
26	Created a nested dictionary	☐	☐
27	Accessed nested dictionary elements	☐	☐
28	Modified nested dictionary information	☐	☐
29	Added information to a nested dictionary	☐	☐
30	Distinguished dictionaries from lists	☐	☐
31	Distinguished dictionaries from tuples	☐	☐
32	Applied dictionaries to structured data storage	☐	☐

No.	Criteria	Yes	No
33	Saved the program with .py extension	☐	☐
34	Executed the Python program successfully	☐	☐
35	Tested dictionary access operations	☐	☐
36	Tested dictionary modification operations	☐	☐
37	Tested dictionary removal operations	☐	☐
38	Tested nested dictionary operations	☐	☐
39	Identified missing-key errors	☐	☐
40	Identified dictionary syntax errors	☐	☐
41	Corrected identified errors	☐	☐
42	Applied appropriate debugging techniques	☐	☐
43	Verified final dictionary contents	☐	☐
44	Verified final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____